

Corner Cover

Evirir has a $n \times m$ grid. The cell in row i and column j is called (i, j) . The *corners cells* are the cells $(1, 1)$, $(1, m)$, $(n, 1)$, and (n, m) .

Evirir wants to choose a $A \times B$ or $B \times A$ subgrid in the grid such that the subgrid contains **at least** two corner cells. Is this possible?

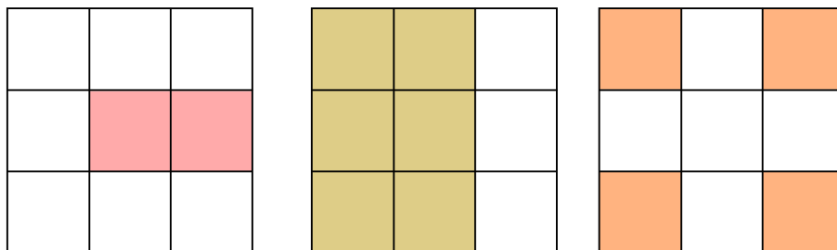


Figure 1: Illustration of two examples of subgrid as well as showing the corner cells

The first two diagrams show some of the possible subgrids (1×2 and 3×2 respectively), and the rightmost diagram shows the 4 corner cells in the grid. The first diagram does not cover two corner cells, but the second one does.

Input Format

There are multiple test cases. The first line contains the number of test cases T . Each of the next T lines contains a test case. The description of a test case follows.

The only line of each test case contains four space-separated integers, n , m , A , and B ($2 \leq n, m, A, B \leq 10^{18}$).

Output Format

Output T lines. For each test case, output YES on a line if it is possible and output NO otherwise.

Constraints

Task	1	2	3	4	5
t	(samples)	1	200	200	200
n, m, A, B	(samples)	10	10	10^9	10^{18}
Points	0	10	10	40	40

Sample Input

3

```

2 3 3 3
2 3 2 3
2 3 2 1

```

Sample Output

```

NO
YES
YES

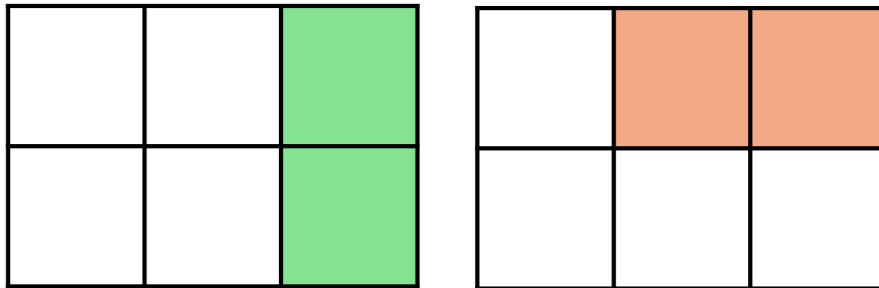
```

Sample Explanation

In the first test case, it is impossible to choose a 3×3 subgrid from a 2×3 .

In the second test case, it is possible by choosing the entire grid.

In the third test case, one can cover 2 corner cells with a 2×1 subgrid as shown on the left. Note that it is also allowed to use a 1×2 subgrid as shown on the right, but the right subgrid only covers one corner cell and not at least two.



Gifts

You are hosting a party with n guests but have prepared only m gifts, where $m < n$. To decide who should receive a gift, you assign each guest a tier number t_i ($1 \leq t_i \leq 10^9$). The lower the tier number, the closer the guest is to you. Guests with a tier number of 1 are your closest friends.

You will distribute the gifts based on the tier numbers:

- First, give gifts to all guests in tier 1.
- Then, proceed to tier 2, tier 3, and so on.
- If there are not enough gifts for all guests in a tier, prioritize guests based on their arrival order. For simplicity, guests arrive in the order of their indices; that is, guest i arrives before guest j if $i < j$.

Determine which guests will receive a gift.

Input Format

- The first line contains two integers n and m ($1 \leq m < n \leq 10^5$) — the number of guests and the number of gifts.
- The second line contains n integers $t_1, t_2, \dots, t_n$ ($1 \leq t_i \leq 10^9$) — the tier numbers of the guests.

Output Format

Output a single line containing n integers $x_1, x_2, \dots, x_n$, where $x_i = 1$ if guest i receives a gift, and $x_i = 0$ otherwise.

Sample Input

```
8 6
3 1 4 1 5 9 2 5
```

Sample Output

```
1 1 1 1 1 0 1 0
```

Explanation

- **Guests 2 and 4** are in tier 1 and both receive gifts.
- **Guest 7** is in tier 2 and receives a gift.
- **Guest 1** is in tier 3 and receives a gift.
- **Guest 3** is in tier 4 and receives a gift.
- **Guest 5 and 8** is in tier 5 but there is only one gift left. Since $5 < 9$, **Guest 5** receives a gift but not **Guest 8**.
- **Guest 6** is in tier 9 and does not receive a gift.

Constraints

- $1 \leq m < n \leq 10^5$,
- $1 \leq t_i \leq 10^9$ for all $1 \leq i \leq n$

Magical Orbs

There are n magical orbs, the i -th orb has magical power a_i . Evirir the dragon wizard wishes to combine all orbs to one super mega ultra magical orb. To do so, Evirir the dragon can do the following operation while there is more than one magical orb:

- Pick two orbs with power x and y .
- Destroy the two orbs and make a new orb with magical power $x + 2y$.

As Evirir the dragon did not graduate from wizard school, he needs your help to find the largest magical power the combined orb can have.

Input Format

There are multiple test cases. The first line contains the number of test cases T . Each of the next T lines contains a test case. The description of a test case follows.

The first line of each test case contains a single integer n . The second line contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 10^{18}$) denoting the powers of the orbs.

It is guaranteed that the sum of n over all testcases is at most $2 \cdot 10^5$.

Output Format

Output T lines. For each test case, output the biggest magical power you can achieve modulo $10^9 + 7$.

Note: Modulo means “taking the remainder of”, you have to output the remainder of the answer after dividing by $10^9 + 7$.

Constraints

Task 1: samples (0 points) Task 2: $n, T \leq 10$ (10 points) Task 3: $a_1 = a_2 = \dots = a_n$ and $a_i \leq 10^9$ (10 points) Task 4: $a_i \leq 10^9$ (20 points) Task 5: no constraints (20 points) Task 6: no constraints (20 points) Task 7: no constraints (20 points)

Sample Input

```
2
3
1 2 1
4
1 2 3 2
```

Sample Output

```
11
```

Sample Explanation

In the first test case, we merge orb 3 (power $x = 1$) and orb 2 (with power $y = 2$), making a new orb with power $x + 2y = 1 + 2 \cdot 2 = 5$. Then, we can merge orb 1 (with power $x = 1$) with the new orb (with power $y = 5$) to get a new orb with power $1 + 2 \cdot 5 = 11$.

Simple Game

Evirir and Rhae are playing a game. Initially, there are n pairs of integers $(a_1, b_1), \dots, (a_n, b_n)$ and both players have 0 points. Evirir and Rhae take turns to act, with Evirir going first.

On Evirir's turn, they will choose a pair (a_i, b_i) that has not been removed, add a_i to Evirir's total score, and remove the pair (a_i, b_i) .

On Rhae's turn, they will choose a pair (a_i, b_i) that has not been removed, add b_i to Rhae's total score, and remove the pair (a_i, b_i) .

The game ends when all pairs have been removed. Let X and Y be the final score of Evirir and Rhae, respectively. Evirir wants to maximize the value of $X - Y$ and Rhae wants to minimize it. If both players play optimally, what is the final value of $X - Y$?

Input Format

The first line contains one integer, n ($2 \leq n \leq 10^4$), the total number of pairs.

The next n lines contains two space-separated integers, a_i and b_i ($-10^9 \leq a_i, b_i \leq 10^9$), the values of i^{th} pair.

Output Format

Output a single integer, the final value of $X - Y$ if both players play optimally.

Sample Input

```
2
4 2
1 3
```

Sample Output

```
1
```

Sample Explanation

There are two pairs of integers $(4, 2)$ and $(1, 3)$.

There are only two possible cases: - Evirir takes the first pair of integers and Rhae takes the second pair of integers. The final scores: X is 4 and Y is 3, resulting $X - Y = 1$. - Evirir takes the second pair of integers and Rhae takes the first pair of integers. The final scores: X is 1 and Y is 2, resulting $X - Y = -1$.

Since Evirir wants to maximize $X - Y$ and Evirir has the ability to force Rhae to take the remaining pair, Evirir takes the first pair and Rhae takes the second pair, resulting $X - Y = 1$.

Exploding Arrow

You are given three integers N , M , and K . You have K arrows, and your goal is to destroy N targets. Each target i has an initial hit points (hp) value of a_i . To destroy a target, its hp must be less than or equal to 0.

When you shoot an arrow with power X at target i , it decreases the hp of all targets with index $j \geq i$ by an amount calculated as follows:

$$\text{Damage to target } j = \max(0, M \cdot X - (j - i)^2)$$

Your task is to find the minimum possible value of X such that you can destroy all targets, given that you can choose the indices of your arrows optimally.

Input Format

- The first line contains three integers N , M , and K : the number of targets, the multiplier for the arrow power, and the number of arrows, respectively.
- The second line contains N integers $a_0, a_1, \dots, a_{N-1}$ representing the initial hp of the targets.

Output Format

- Output a single integer, the minimum value of X such that you can destroy all targets.

Constraints

Denote $A = a_0 + a_1 + \dots + a_{N-1}$. In all test cases we have - $1 \leq M \leq 10^9$. - $1 \leq N \leq 2 \times 10^5$. - $1 \leq K \leq 10^9$. - $1 \leq a_i \leq 10^9$.

Constraints for each test case: 1. Sample 2. $K = 1$. 3. $A \leq 10000 \cdot MK$ and $N \leq 100$. 4. $A \leq 5 \times 10^6 \cdot MK$ and $N \leq 5000$. 5. $N \leq 1000$. 6. $N \leq 1000$. 7. $N \leq 2 \times 10^5$. 8. $N \leq 2 \times 10^5$.

Example

Input

```
4 1 2
3 3 2 4
```

Output

```
5
```


Explanation

In the example, you can destroy all targets with the following strategy:

1. Shoot the first arrow (index 0) with power $X = 5$:
 - Damage to target 0: $1 \cdot 5 - (0 - 0)^2 = 5 \rightarrow a[0] = 3 - 5 = -2$
 - Damage to target 1: $1 \cdot 5 - (1 - 0)^2 = 4 \rightarrow a[1] = 3 - 4 = -1$
 - Damage to target 2: $1 \cdot 5 - (2 - 0)^2 = 1 \rightarrow a[2] = 2 - 1 = 1$
 - Damage to target 3: $1 \cdot 5 - (3 - 0)^2 = -4 \rightarrow a[3] = 4 - 0 = 4$
2. Shoot the second arrow (index 2) with power $X = 5$:
 - Damage to target 2: $1 \cdot 5 - (2 - 2)^2 = 5 \rightarrow a[2] = 1 - 5 = -4$
 - Damage to target 3: $1 \cdot 5 - (3 - 2)^2 = 4 \rightarrow a[3] = 4 - 4 = 0$

All targets are eliminated.

XOR the String

A binary string is a string consisting of 0s and 1s. You are given a binary string $s = s_1 s_2 \dots s_n$ of length n and an integer $k \geq 0$.

For a binary string $t = t_1 t_2 \dots t_m$ ($m \geq 2$), **transforming** t means changing t to $t_1 (t_1 \oplus t_2) t_2 (t_2 \oplus t_3) t_3 \dots t_{m-1} (t_{m-1} \oplus t_m) t_m$

where $\oplus$ is the bitwise XOR operator. For example, 011 transformed is 01101, and 10011 transformed is 110001101. Here is an image showing how 10011 is transformed:

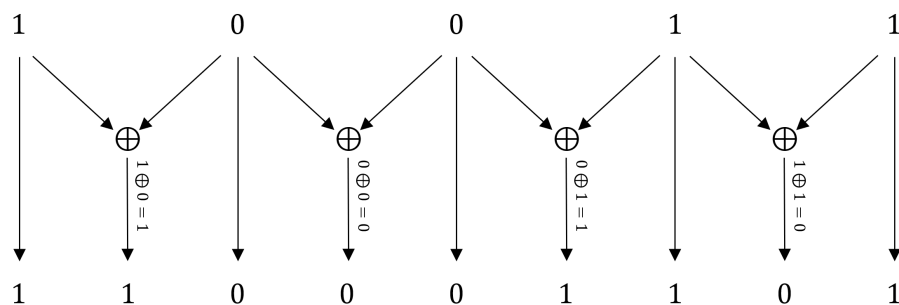


Figure 2: Illustration of how 10011 is transformed into 110001101. Between two characters, the XOR of the characters is inserted between them.

The **beauty** of a binary string $t = t_1 t_2 \dots t_m$ ($m \geq 2$) is the number of integers i ($1 \leq i < m$) such that $t_i = t_{i+1}$. We will denote the beauty of t by $f(t)$. For example, $f(10010111) = 3$, $f(010) = 0$, and $f(000) = 2$. Also, define $g(t)$ to be the value of $f(t')$, where t' is t transformed k times.

Output the sum of $g(t)$ over all substrings of s with length at least 2. In other words, let $s[l, r]$ be the substring $s_l s_{l+1} \dots s_r$. Output $\sum_{1 \leq l < r \leq n} g(s[l, r])$. Since the answer might be very large, output the answer modulo 998244353.

Input Format

The first line contains two space-separated integers, n and k ($2 \leq n \leq 2 \cdot 10^5$, $0 \leq k \leq 10^{18}$).

The second line contains s , a binary string of length n .

Output Format

Output a single integer, the sum of $g(t)$ of all substrings t of s with length at least 2, modulo 998244353.

Constraints

Task	1, 2	3	4	5	6	7	8	9
n	(samples)	5000	15	200000	500	5000	200000	200000
k	(samples)	0	10	0	500	10^{18}	10^6	10^{18}
Points	0	5	10	10	20	20	15	20

Sample Input 1

3 2
101

Sample Output 1

4

Sample 1 Explanation

There are three substrings of length at least 2: 10, 01, and 101. The substrings are transformed $k = 2$ times as follows:

- $10 \rightarrow 110 \rightarrow 10110$ (beauty 1)
- $01 \rightarrow 011 \rightarrow 01101$ (beauty 1)
- $101 \rightarrow 11011 \rightarrow 101101101$ (beauty 2)

The answer is $1 + 1 + 2 = 4$.

Sample Input 2

2 30
00

Sample Output 2

75497471

Sample 2 Explanation

Remember to output the answer modulo 998244353.